Assignment Kit for Coding Standard

Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview

This assignment kit covers the following topics.

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements	Produce, document, and submit a completed coding standard that calls for quality coding practices. For LOC counting, ensure that a separate physical source line is used for each logical line of code. Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example
- Since it is an example, tailor it to meet your personal needs.
- If you have an existing organizational standard, consider using it for the PSP exercises.

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	/**/ /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ /**/
Listing Contents	Provide a summary of the listing contents
Contents Example	/**/ /* Listing Contents: */ /* Reuse instructions */ /* Modification instructions */ /* Compilation instructions */ /* Includes */ /* Class declarations: */ /* CData */ /* ASet */ /* Source code in c:/classes/CData.cpp: */ /* CData */ /* CData() */ /* Empty() */ /**/

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	/***/ /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ /***/
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	If(record_count > limit) /* have all records been processed? */
Bad Comment	If(record_count > limit) /* check if record count exceeds limit */
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	/***/ /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ /***/
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.

Indenting Example	while (miss_distance > threshold)
	{
	success_code = move_robot (target _location);
	if (success_code == MOVE_FAILED)
	{
	printf("The robot move has failed.\n");
	}
	}

Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization Examples	#define DEFAULT-NUMBER-OF-STUDENTS 15 int class-size = DEFAULT-NUMBER-OF-STUDENTS;

Evaluation criteria and suggestions

Evaluation criteria	Your standard must be • complete • legible
Suggestions	Keep your standards simple and short. Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	Guiar la implementación y documentación de programas en Java para el conteo correcto de líneas y métodos.
Program Headers	Comenzar todos los programas con un bloque descriptivo estandarizado.
Header Format	/** * Asignación de Programa: PSP 2A * Nombre: [Emma Hernández Mendoza] * Fecha: [2025-06-11] * Descripción: [Descripción breve de la clase] *
Listing Contents	Provide a summary of the listing contents.
Contents Example	
Reuse Instructions	• Utilizar comentarios Javadoc antes de cada método para explicar su propósito, parámetros y retorno
Reuse Example	/** * Lee el contenido de un archivo y lo retorna como String * @param inFile nombre del archivo a leer * @return contenido del archivo como String, o string vacío en caso de error */ public String readData(String inFile) { ... }
Identifiers	Usar nombres descriptivos en inglés o combinados, utilizando CamelCase. Evitar nombres de una sola letra a menos que sea un contexto obvio.
Identifier Example	• Good: fileContent, linesArray, trimmedLine, methodCount. • Bad: x, s, d.

(continued)

Coding Standard Template (continued)

Comments	• Documentar el código línea por línea o por bloques lógicos para explicar el "qué" y el "por qué" de la operación, especialmente dentro de bucles y condiciones.
Good Comment	// Verificar si la línea debe ser excluida del conteo if (trimmedLine.isEmpty() || ...
Bad Comment	// Verificar línea if (trimmedLine.isEmpty() || ...
Major Sections	Precede major program sections by a block comment that describes the processing that is done in the next section
Example	
Blank Spaces	• Utilizar líneas en blanco para separar la declaración de variables de la lógica y para separar bloques lógicos distintos (como la lectura de archivos del procesamiento).
Indenting	• Indentar todo el código dentro de las llaves de clase y método. Usar 4 espacios para la indentación estándar.

Indenting Example	public class LineCounter {
	public int count(String[] artData) {
	int count = 0;
	for (String line : artData) {
	// lógica
	}
	return count;
	}
	}

Capitalization	• Clases: PascalCase (ej. LineCounter, Logic2a). • Métodos y Variables: camelCase (ej. readData, fileName).
Capitalization Example	Capitalization Example Logic2a, LineCounter, App (Clases) readData, fileName, myMethodCounter, saveData (Variables y Métodos)

